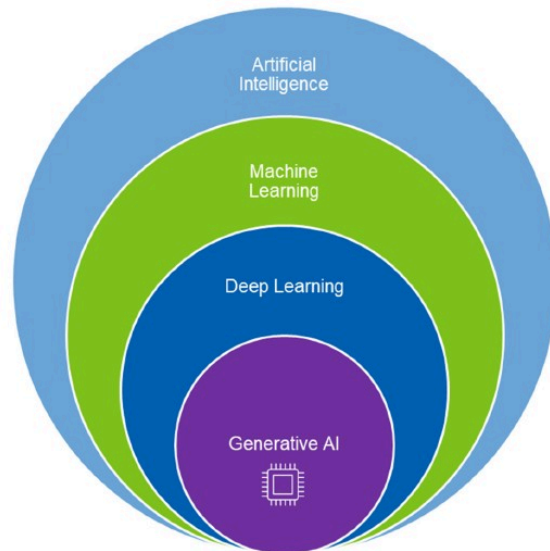


Tema 9 Introducción a IA Generativa

Autor: Ismael Sagredo Olivenza

8.1 ¿Qué es la IA Generativa?

La IA generativa es una subcategoría de la IA entrenada con una gran variedad de datos, que aprende los patrones y distribuciones subyacentes. La magia reside en su potencial para generar algo «novedoso» y «original», una tarea que antes se creía que era dominio exclusivo del ingenio humano.



8.1.1 ¿Que cosas podemos generar?

- Extracción de entidades: Fragmentos de información que nos interesan usando LLMs sustituyendo a los tradicionales Named Entity Recognition (NER).
- Generación de texto: Desde texto simple a poemas, bromas, noticias, traducciones, etc. Se usan LLMs
- Generación de imágenes: Genera imágenes a partir de un texto descriptivo de objetos que como tal no existen en el mundo real. Aplica diferentes estilos a las imagenes (de dibujos, de un pintor famoso...)

- Generación de música: Los modelos de IA generativa se están utilizando para crear música nueva que es original y creativa. Esta tecnología se utiliza para diversos fines, como crear música para películas y programas de televisión.
- Generación de código: asistentes de programación (Co-pilot) generación de plantillas, documentación...
- Generación de video: Generar video realista es una extensión de las capacidades de generar imágenes con la peculiaridad de que mantienen la lógica de la secuencia que se está generando.

8.1.2 Diferencia entre IA tradicional y generativa

- **La IA tradicional (Narrow):** Se utiliza para predecir cosas que operan dentro de los límites preestablecidos para los que ha sido entrenada. Estos límites son las reglas e instrucciones que están codificadas en un modelo. Solo puede actuar basándose en condiciones predefinidas, restricciones y resultados potenciales. El resultado, por lo tanto, es determinista y relativamente predecible.
- **IA Generativa:** El resultado es probabilístico influenciadas por los datos de entrada y los patrones aprendidos. Lo que permite es generar contenido no directamente aprendido (no se había enseñado explícitamente al sistema). El entrenamiento de la IA generativa se apoya en la IA tradicional y ambas se complementan.

8.2 self-supervised learning

Es un híbrido entre aprendizaje supervisado y no supervisado. Utiliza las técnicas de entrenamiento del aprendizaje supervisado, pero con datos sin etiquetar de forma que es capaz de resolver problemas no supervisados. Se puede considerar como aprendizaje supervisado sin intervención humana. Las "etiquetas" existen, pero estas se generan a partir de los datos de entrada, normalmente utilizando un algoritmo heurístico. Ejemplos: Autoencoders, predicción de la siguiente palabra o del siguiente fotograma (en estos dos casos se denomina supervisión temporal)

8.3 Redes generativas adversarias (GANs)

Las redes adversarias generativas (GANs) son un método basado en el entrenamiento de dos redes neuronales, una denominada **generadora** y otra **discriminadora**, compitiendo entre sí para generar nuevas instancias que se asemejen a las de la distribución de probabilidad de los datos de entrenamiento. Se han utilizado ampliamente en:

- Generación de imagen a partir de texto
- Síntesis de series temporales
- Visión por computador

8.3.1 Fundamento matemático

Para cualquier conjunto de datos, podemos hipotetizar que es posible definir una distribución de probabilidad P_{data} representativa de la población representada por la muestra formada por el conjunto de datos. De ser esto posible, para cualquier valor de x será posible establecer un valor $P_{data}(x)$ que determine la probabilidad de que x pertenezca a la población.

Una vez hayada esa distribución de probabilidad, dada una instancia podríamos saber la probabilidad de pertenecer a la población.

Ejemplo: la probabilidad de que un individuo cualquiera sea un estudiante universitario.

Un modelo generativo modeliza la probabilidad dentro de su modelo y permite generar variaciones aleatorias proximas al modelo.

Ejemplo: aprendo lo que es un Van Gogh y genero versiones ligeramente modificadas de un Van Gogh.

Pero no son capaces por si mismas de indicar cual es la probabilidad de dada una instancia, que esta pertenezca a la población.

Las redes GAN son capaces de codificar estas distribuciones de probabilidad.

8.3.2 Redes antagónicas

La arquitectura GAN está compuesta por dos redes la red **discriminante D** y la red **generadora G**.

- G genera nuevas instancias del mismo dominio que el conjunto de datos
- D discrimina si los datos de entrada son reales, es decir, si pertenecen al conjunto de entrenamiento o no.

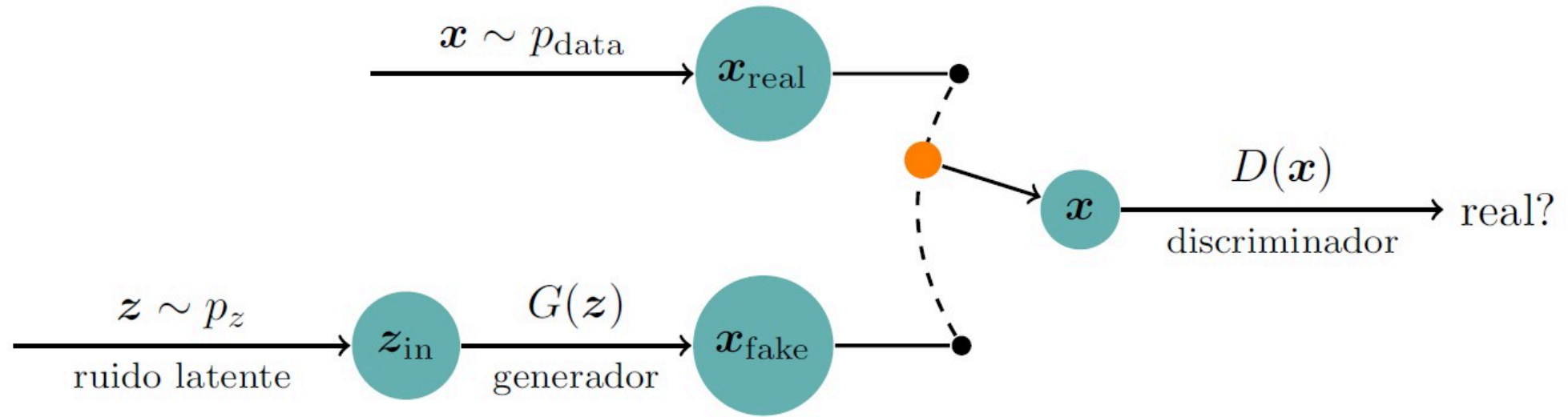
Ambas redes se entrenan de forma conjunta. G debe maximizar sus posibilidades de no ser detectada por D es cada vez más capaz de detectar un posible fraude en los datos generados. Compitiendo ambas en un juego de **suma cero** llegando a un equilibrio.

8.3.2.1 Como funciona una GAN

Supongamos un modleo de generaci3n de im3genes.

A la red D (discriminadora) le llegan a veces valores de la poblaci3n de muestra (imagenes del dataset) y a veces valores generados artificialmente con G. D debe discriminar si se trata de un valor real o un valor ficticio.

G debe generar valores cada vez mas parecidos a los reales para intentar engañar a D.



- $X \sim P_{\text{data}}$ sigue la distribución de probabilidad (son imágenes reales)
- $Z \sim P_z$ es ruido y a partir de ese ruido $G(z)$ es capaz de generar valores cada vez más parecidos a $X \sim P_{\text{data}}$.
- El vector z pertenece a una distribución de probabilidad aleatoria P_z .

funciones de optimización

$$\min(G), \max(D) = Y(x) * \text{Log}(D(x)) + Y(z) * \text{Log}(1 - D(G(x)))$$

Es decir, la función min max a optimizar es la salida real por el logaritmo de la salida del discriminador con los datos reales + la salida falsa por 1 - la salida del discriminador procesando la entrada falsa (la generada por G)

Las funciones a optimizar son la **Cross Entropy** a veces se utiliza una variante **BCEWithLogitsLoss** que es numericamente más estable (menor pérdida del gradiente).

```

class Generator(nn.Module):
    def __init__(self, noise_dim):
        super(Generator, self).__init__()
        self.noise_dim = noise_dim
        self.main = nn.Sequential(
            nn.Linear(noise_dim, 7 * 7 * 256),
            nn.ReLU(True),
            nn.Unflatten(1, (256, 7, 7)),
            nn.ConvTranspose2d(256, 128, 5, stride=1, padding=2),
            nn.BatchNorm2d(128),
            nn.ReLU(True),
            nn.ConvTranspose2d(128, 64, 5, stride=2,
                               padding=2, output_padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(True),
            nn.ConvTranspose2d(64, 1, 5, stride=2,
                               padding=2, output_padding=1),
            nn.Tanh()
        )

    def forward(self, x):
        return self.main(x)

```

```
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.main = nn.Sequential(
            nn.Conv2d(1, 64, 5, stride=2, padding=2),
            nn.LeakyReLU(0.2, inplace=True),
            nn.BatchNorm2d(64),
            nn.Conv2d(64, 128, 5, stride=2, padding=2),
            nn.LeakyReLU(0.2, inplace=True),
            nn.BatchNorm2d(128),
            nn.Flatten(),
            nn.Linear(7 * 7 * 128, 1)
        )

    def forward(self, x):
        return self.main(x)
```

Combinando ambas

```
NOISE_DIM = 100

generator = Generator(NOISE_DIM)
discriminator = Discriminator()
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
generator = generator.to(device)
discriminator = discriminator.to(device)

criterion = nn.BCEWithLogitsLoss()

generator_optimizer = optim.Adam(
    generator.parameters(), lr=0.0002, betas=(0.5, 0.999))
discriminator_optimizer = optim.Adam(
    discriminator.parameters(), lr=0.0002, betas=(0.5, 0.999))

NUM_EPOCHS = 5
BATCH_SIZE = 256
```



```

for epoch in range(NUM_EPOCHS):
    for i, data in enumerate(train_loader):
        real_images, _ = data
        real_images = real_images.to(device)
        #entrenamos el discriminador con las imagenes reales
        discriminator_optimizer.zero_grad()
        real_labels = torch.ones(real_images.size(0), 1, device=device)
        real_outputs = discriminator(real_images)
        real_loss = criterion(real_outputs, real_labels)
        real_loss.backward()
        #entrenamos el discriminador con la imagenes falsas
        noise = torch.randn(real_images.size(0), NOISE_DIM, device=device)
        fake_images = generator(noise)
        fake_labels = torch.zeros(real_images.size(0), 1, device=device)
        fake_outputs = discriminator(fake_images.detach())
        fake_loss = criterion(fake_outputs, fake_labels)
        fake_loss.backward()
        discriminator_optimizer.step()
        #entrenamos el generador
        generator_optimizer.zero_grad()
        fake_labels = torch.ones(real_images.size(0), 1, device=device) # son 1 porque son falsas
        fake_outputs = discriminator(fake_images)
        gen_loss = criterion(fake_outputs, fake_labels) #el discriminador como las clasifica?
        gen_loss.backward()
        generator_optimizer.step()

    if i % 100 == 0:
        print(f'Epoch [{epoch+1}/{NUM_EPOCHS}], Step [{i+1}/{len(train_loader)}], '
              f'Discriminator Loss: {real_loss.item() + fake_loss.item():.4f}, '
              f'Generator Loss: {gen_loss.item():.4f}')

```

- BCEWithLogitsLoss: Binary Cross Entropy con Logits Loss es mas estable que un BCE tradicional
- zero_grad() garantiza que los gradientes de iteraciones anteriores no interfieran con el paso de entrenamiento actual.
- real_labels = torch.ones : etiqueta las labels como 1 correctas.
- noise = torch.randn Genera numeros aleatorios.
- fake_labels = torch.zeros sabemos que son falsas.
- fake_loss = criterion(fake_outputs, fake_labels)
Cross entropy entre la salida estimada outputs y la real labels.
- fake_labels = torch.ones en el generador. Son clase 1 porque son falsas hay que compararlo con como las clasifica el discriminador (se debe equivocar y decir que son verdaderas)

8.3.2.2 Utilización una vez entrenada.

El generador una vez entrenado es capaz de generar a partir de un dato nuevo aleatorio una imagen (si procesamos imágenes) similar a los datos aprendidos.

Si por ejemplo lo hemos entrenado con datos de fotos de anime, será capaz de generarnos una nueva imagen de anime.

Usos:

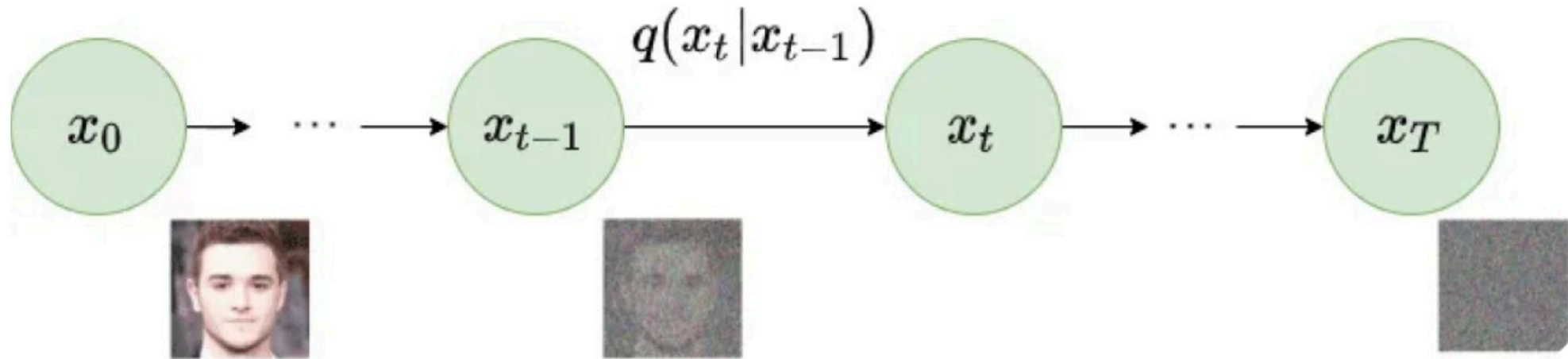
- Data augmentation
- Cambios de estilo
- Síntesis de voz, imágenes...

8.4 Modelos de difusión

Los modelos de difusión realizan modificaciones a una imagen sucesivas añadiendo ruido a la misma. Funcionan de forma similar a las GAN ya que se entrenan dos redes, una es capaz de reconstruir la imagen a partir del ruido y la otro de generar ruido a partir de la imagen. Utilizan una distribución de probabilidad de Markov que recordemos, en cada paso la probabilidad del siguiente estado depende del estado anterior.

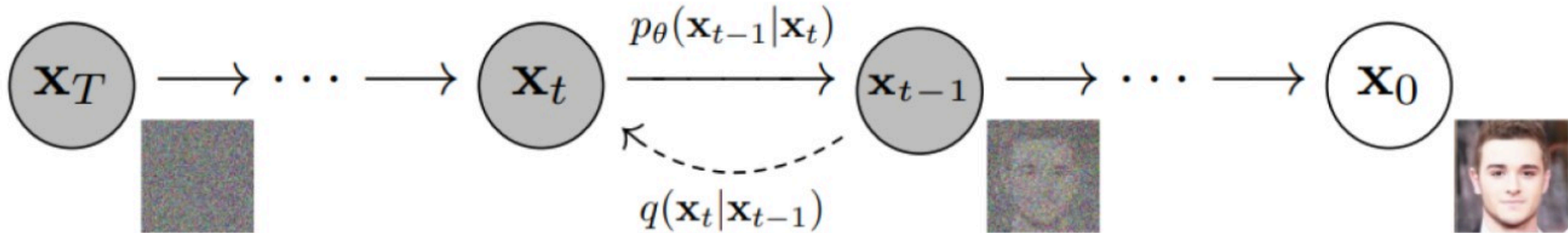
8.4.1 Difusion hacia delante:

El modelo transforma gradualmente la imagen en una serie de pasos. La imagen comienza siendo clara y se vuelve cada vez más ruidosa a medida que avanza en cada paso, hasta convertirse en un ruido casi completo al final.



8.4.2 Difusión hacia atrás:

El proceso de difusión inversa comienza una vez que el proceso de difusión directa ha transformado una muestra en un estado ruidoso y complejo. Mapea gradualmente la muestra ruidosa de nuevo a su estado original utilizando una serie de transformaciones inversas. Los pasos que invierten el proceso de adición de ruido están guiados por una Cadena de Markov inversa.



8.4.3 Usos

- Generación de imágenes a partir de texto (Stable Diffusion, Dall-E) interviene un modelo de lenguaje.
- Geenración de música

8.5 Transformer

Son la base de los modelos fundamentales desde los que surgen los LLMs más utilizados actualmente. El modelo Transformer se presentó por primera vez en el artículo «Attention is All You Need»

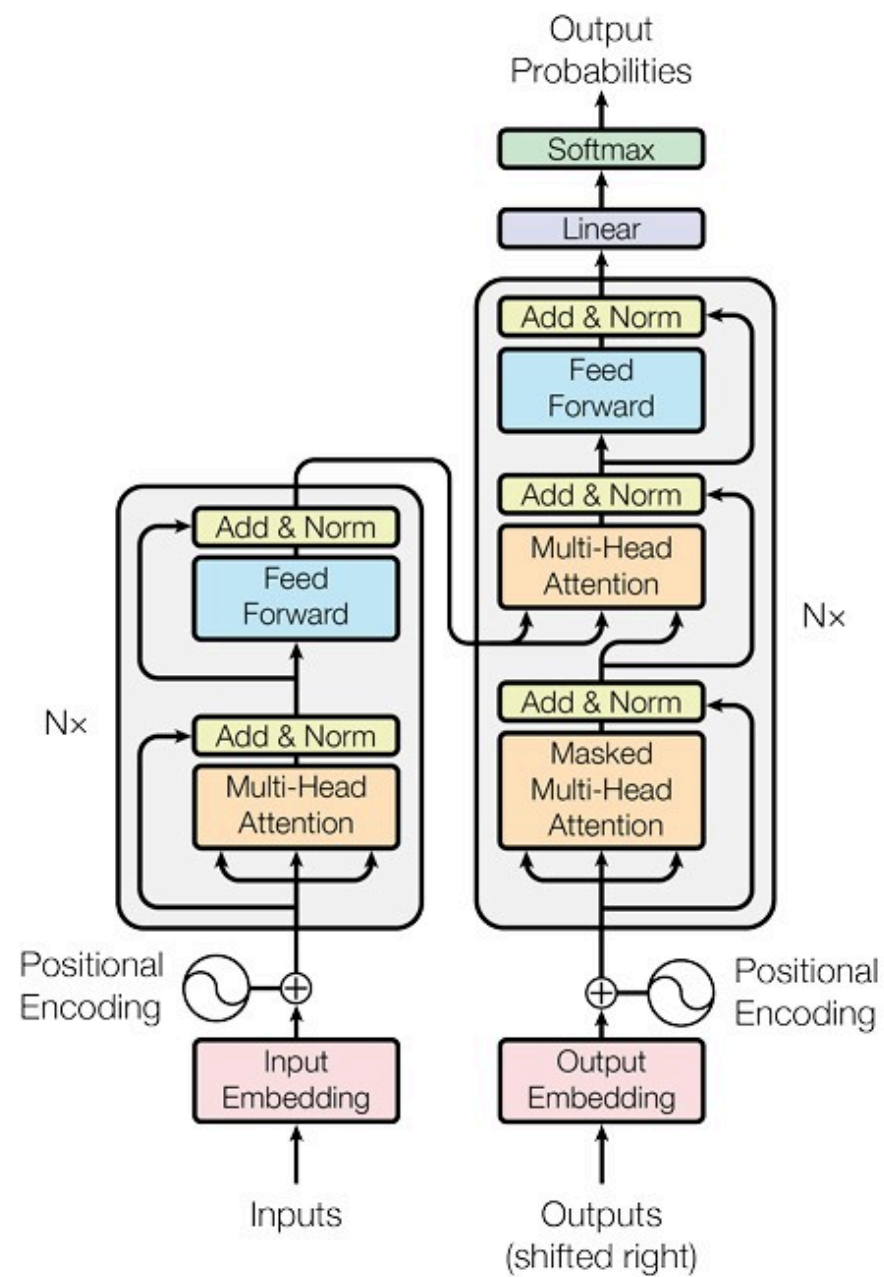
<https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>

Utilizan un mecanismo conocido como self-attention que permite al modelo tener en cuenta todo el contexto de una frase, considerando todas las palabras simultáneamente en lugar de procesar la frase palabra por palabra. Este enfoque es más eficiente y puede mejorar los resultados en muchas tareas de PLN.

La ventaja es que capta las dependencias independientemente de su posición en el texto.

Esto es fundamental para tareas como la traducción automática y el resumen de textos, en las que el significado de una frase puede depender de palabras que se encuentran a varias palabras de distancia.

Son altamente paralelizables por lo que son mas rapidos de entrenar que otros modelos.



Puntos clave

- **Arquitectura codificador-decodificador.**

El codificador mapea una secuencia de entrada de representaciones de símbolos $(x_1; \dots; x_n)$ a una secuencia de representaciones continuas $z = (z_1; \dots; z_n)$

Dado z , el decodificador genera entonces una secuencia de salida $(y_1; \dots; y_m)$ de símbolos, un elemento a la vez. En cada paso, el modelo es auto-regresivo, consumiendo los símbolos previamente generados como entrada adicional al generar el siguiente.

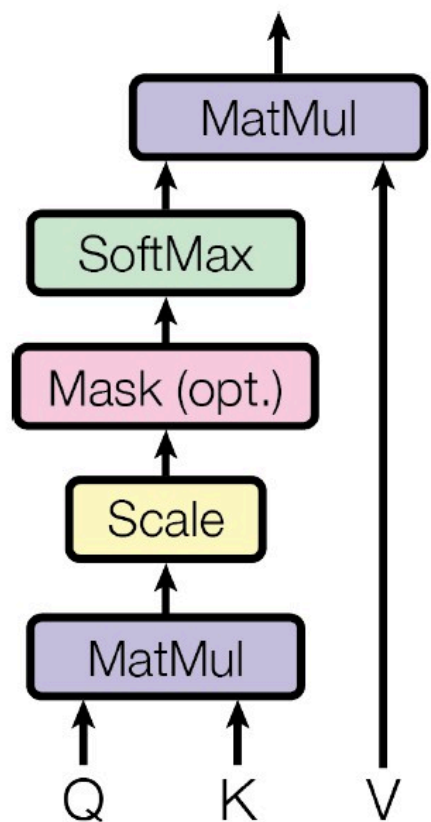
- **Capas de atención apiladas (stacked self-attention) y capas full connect.**

Encoder: Compuesto por una pila de $N = 6$ layer idénticas, cada una de ellas dividida en dos sub layeres. La primera es una capa denominada multi-head self-attention y la segunda una red fully connected feed-forward.

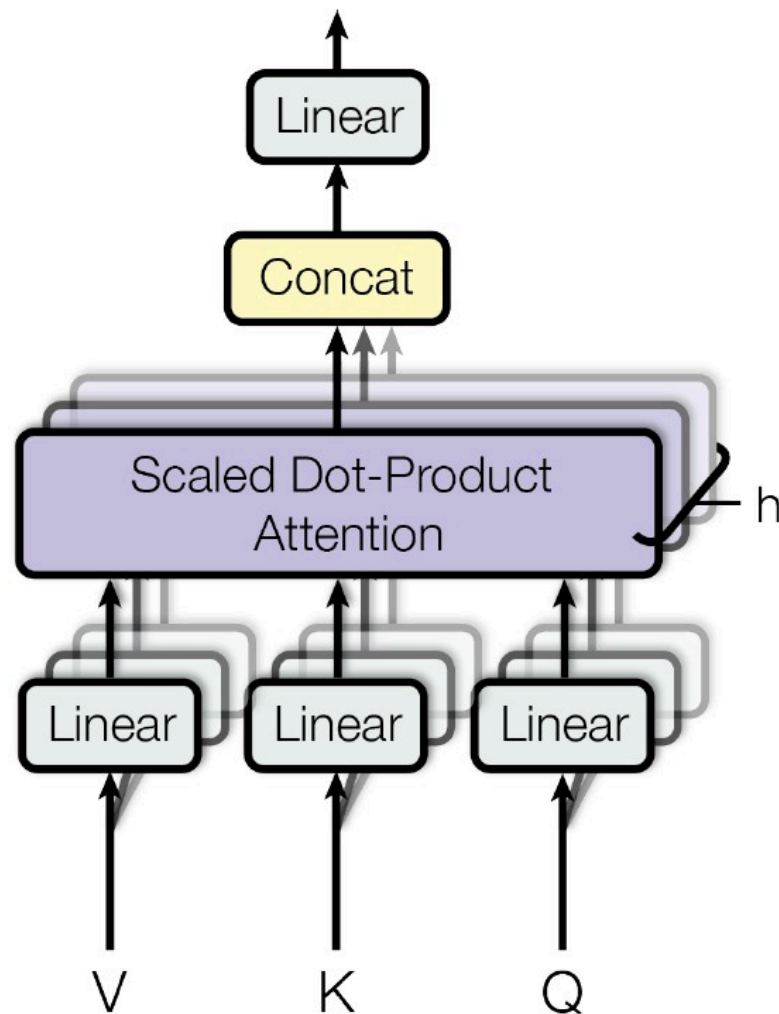
Decoder: también está compuesto por una pila de $N=6$ capas idénticas. Además de las dos subcapas presentes en cada capa del codificador, el decodificador inserta una tercera subcapa, la cual realiza atención multi-cabeza sobre la salida de la pila del codificador.

Multi-Head Attention

Scaled Dot-Product Attention



Multi-Head Attention



La entrada consiste en consultas (queries) y claves (keys) de dimensión dk , y valores (values) de dimensión dv . Calculamos los productos escalares de la consulta con todas las claves, dividimos cada uno por $\sqrt{dk}$ y se aplica una función softmax para obtener los pesos sobre los valores. Los valores se agrupan en las matrices Q de las queries y las claves y los valores en las matrices K y V .
Aplicando la función Softmax

$$attention(Q, K, V) = softmax\left(\frac{QK^t}{\sqrt{d_k}}\right)V$$

Multi atención

En lugar de realizar una única función de atención con claves, valores y consultas de dimensión se proyectan linealmente las consultas, claves y valores h veces mediante diferentes proyecciones lineales aprendidas. En cada una de estas versiones proyectadas de consultas, claves y valores, se ejecuta la función de atención en paralelo, obteniendo valores de salida.

Esto permite captar la atención en múltiples direcciones.

Positional Encoding aprovechar el orden de la secuencia se inyecta cierta información sobre la posición relativa o absoluta de los tokens

8.6 Large Language Models (LLMs)

Los modelos de lenguaje grandes (LLM) son modelos de IA generativa que pueden comprender y generar texto similar al humano a partir de una entrada determinada. Los LLM son la base de muchas tareas de procesamiento del lenguaje natural (NLP), como la búsqueda, la conversión de voz a texto, el análisis de sentimientos, el resumen de textos y mucho más.

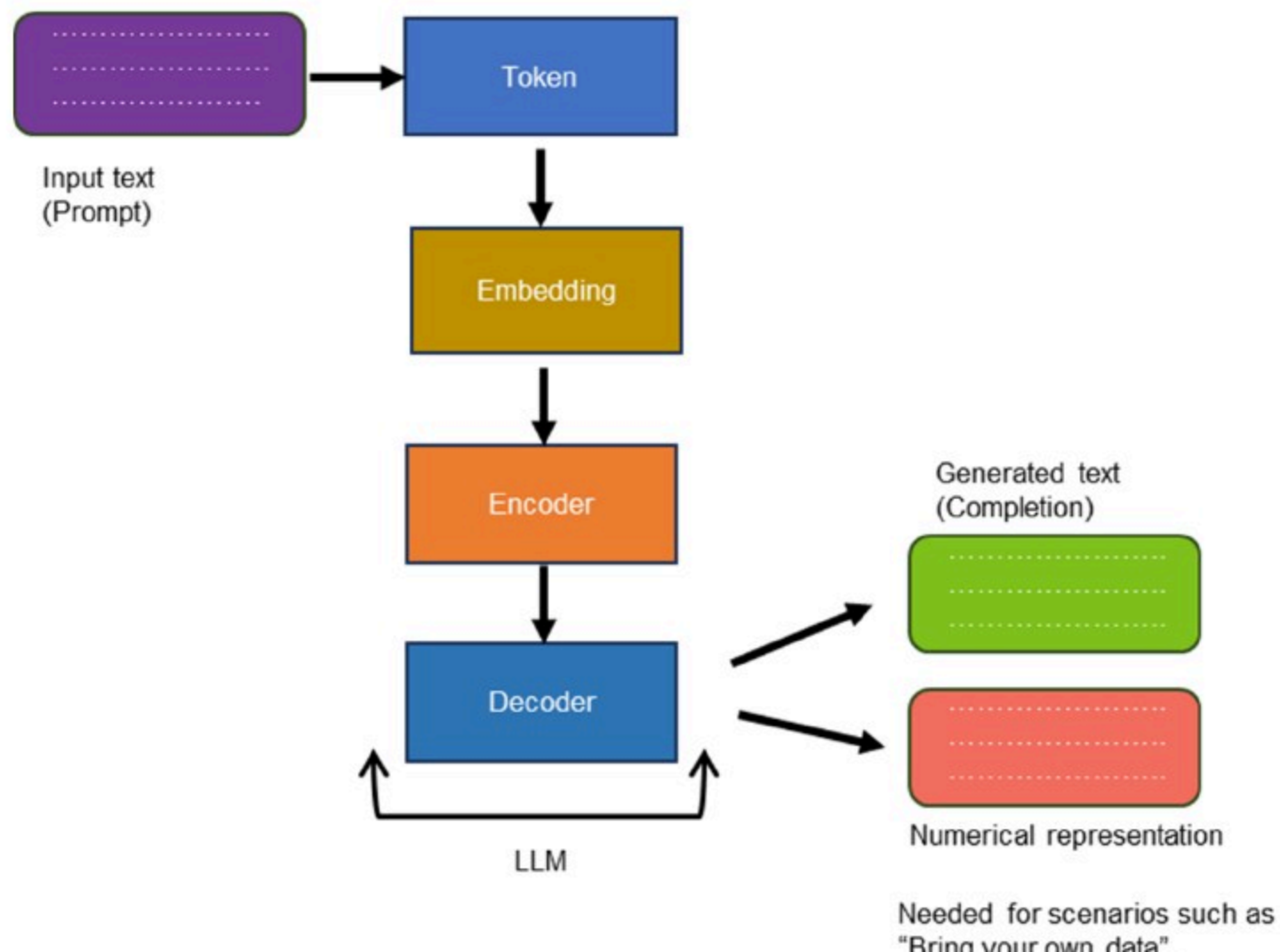
Usan como modelo una red Transformer junto con un sistemas de entrenamiento **aprendizaje por refuerzo humano**, que permite refinar el tipo de modelo que aprende.

Modelos fundamentales

Son aquellos desde los que parten las diferentes aplicaciones. Son los modelos base entrenados con texto del os que luego parten las aplicaciones comerciales como ChatGPT. Algunos de ellos:

- GPT (Generative Pre-trained Transformer):
- Codex: Específicamente creado con código.
- Claude:
- BERT (Bidirectional Encoder Representations from Transformers):
Creado por google.
- PaLM: Es multi-modal (texto, imagene, etc) creado por Google.
- LaMDA: Para chartbots. Creado por google.

8.6.1 Como funciona



El texto de la entrada se convierte a tokens. Cada token se convierte a una representación numérica denominada embedding (incrustaciones).

El codificador (parte codificador del Transformer) procesa la entrada y genera una secuencia de espacios ocultos.

Los espacios ocultos se introducen en el decodificador, con un primer token. El decodificador genera el siguiente token en base a los tokens anteriores y el modelo oculto. Esto se repite y se va convirtiendo el token en texto y es la salida que vemos en el LLM.

8.6.2 Prompt

Es el texto que describe la tarea que queremos realizar utilizando lenguaje natural. Según como realicemos este prompt podemos forzar al LLM a funcionar de una forma concreta. A esto se le denomina **prompt engineering**

El prompt se usa de contexto para generar los siguientes tokens. Como hay una parte de aleatoriedad no genera siempre el más predecible si no que a veces saca otro que a su vez provoca un cambio en la probabilidad del siguiente.

Los LLMs tienen una **ventana de contexto** que es el número máximo de tokens que puede procesar.

8.6.3 Embeddings

Capturan similitudes semánticas en un espacio vectorial. Es decir los embeddings son una colección de vectores semánticos.

La idea detrás de los embeddings es que las palabras con significados similares deben tener representaciones vectoriales similares, medidas por sus distancias. Los vectores con distancias más pequeñas entre ellos sugieren que están muy relacionados, y los que tienen distancias más largas sugieren una baja relación.

Los embeddings se aprenden en base al contexto de las palabras dentro de los textos. Hay diferentes formas de aprenderlos como Similarity embeddings, text search embeddings, etc.

8.6.3 Parámetros de configuración del modelo

- Max Response: Establezca un límite en el número de tokens por respuesta del modelo.
- Temperature: Controla la aleatoriedad. Al reducir la temperatura, el modelo produce respuestas más repetitivas y determinísticas.
- Top P: Al reducir el Top P, se limita la selección de tokens del modelo a aquellos más probables y se ignora la larga cola de tokens menos probables. No es recomendable tocar temperatura y top p a la vez.
- Stop sequences: Fuerza a que el modelo termine de una forma predeterminada con una secuencia de parada (no la incluye en la salida)

- Frequency penalty: Reducir la probabilidad de repetir un token de forma proporcional en función de la frecuencia con la que ha aparecido en el texto. Esto disminuye la probabilidad de repetir el mismo texto en la respuesta.
- Presence penalty: Reducir la probabilidad de repetir cualquier token que haya aparecido en el texto hasta el momento. Esto aumenta la probabilidad de introducir nuevos temas en una respuesta.

8.6.4 Context Window

El rango de tokens o palabras que rodean a una palabra o token en particular que un LLM tiene en cuenta al realizar predicciones. La ventana de contexto ayuda al modelo a comprender las dependencias y relaciones entre las palabras, lo que le permite generar predicciones más precisas y coherente. Pero una ventana de contexto grande hace que el modelo tarde más en computar.

8.6.5 Retrieval-augmented generation (RAG)

Extiende las capacidades de los LLM a dominios específicos o a la base de conocimientos interna de una organización, sin la necesidad de volver a entrenar el modelo. Esto no permite superar la ventana de contexto y almacenar información actualizada del sistema.

Antes de enviar la consulta al LLM se recupera del RAG una serie de bloques (chunks) parecidos semánticamente a los tokens de entrada (usando embeddings).

Para llenar el Rag hay que condicionar el texto nuevo como embeddings y dividirlo en fragmentos (chunks) que después se recuperan e inyectan en el LLM junto al prompt.

8.6.6 algunos LLMs

Comerciales

- ChatGPT
- DeepSheek
- Mistral

Open Source

- LLama (Meta)
- Doll (Databricks)
- Alpaca